

Aangan — Install Guide & Idea Menu

Nathaniel School of Music · 29 Jul 2026 · Part 1: House Pi from scratch · Part 2: the two door screens · Part 3: Music Pi pointer · Part 4: every creative idea, studio + music, with what it becomes in the app

Status: the Aangan app is LIVE at studio-command.vercel.app with the state dial, Pre-flight, Displays + delivery OTP, Family SOS ([/#/sos](#)) and the close-the-door nudge — running in simulation until the House Pi below goes live. Repo: github.com/jasonzacmusic/aangan.

PART 1 — House Pi from scratch Home Assistant OS · ~2 hours hands-on, sensors over a weekend

1 Flash Home Assistant OS.

On the Mac: install [Raspberry Pi Imager](#) → CHOOSE OS → *Other specific-purpose OS* → *Home assistants and home automation* → *Home Assistant* → pick Raspberry Pi 5 → write to a 64GB A2 card. No settings needed — HA configures itself.

2 First boot.

Card into the new Pi 5, Ethernet cable to the router if possible, power with the official 27W PSU. Wait ~5 minutes, then open `http://homeassistant.local:8123` on the Mac. Create the admin account, name the home **Aangan**, set location Bangalore.

3 Install the ESPHome add-on.

Settings → Add-ons → Add-on store → **ESPHome Device Builder** → Install → Start (turn on “Show in sidebar”).

4 Flash the six ESP32 zone nodes.

The six YAML files live in the repo at `pi/house/esphome/`. Copy `secrets.example.yaml` → `secrets.yaml` (Wi-Fi name/password + one generated key per node). First time: plug each ESP32 into the Mac by USB and flash from the ESPHome page; after that all updates are over Wi-Fi. Adopt each node under Settings → Devices & Services.

5 Install the Aangan package.

Copy `pi/house/homeassistant/packages/studio_command.yaml` to `/config/packages/` (use the File editor add-on), add the `packages:` include to `configuration.yaml` as written in `pi/house/README.md`, fix any entity names that adopted differently, restart HA. This creates **binary_sensor.studio_ready** — the one authoritative go/no-go verdict.

6 Family phones.

Everyone (Amma, Jason, brother) installs the **Home Assistant Companion** app and logs into `http://homeassistant.local:8123` on the home Wi-Fi. iPhones: when asked, ALLOW **Critical Alerts** — that is what makes SOS ring through silent mode. Add each phone to the `notify:` group in the package.

7 The wrapper (feeds the app).

Run `pi/house/wrapper/studio_wrapper.py` on any always-on LAN machine (the Piano Pi is fine) on port **8126**, with a long-lived HA token. Then in the app repo set `USE MOCK = false` and the wrapper URL in `src/config.ts`. Claude does this step.

The wrapper now also implements the `/api/sos` endpoints (contract in the repo README) so the Family SOS is real: hold on any phone → HA critical alerts + signs flash + panels take over.

8 Sensors, safety-first order.

① Certified smoke + LPG alarms mounted and tested standalone FIRST. ② Door reeds on the 4 double doors — at the **meeting stiles (middle)**: switch on one leaf, magnet on the other, so either leaf opening trips it. ③ MQ-6 near each stove, flame sensors, leak probes at every sink/stove line. ④ The panic-loop and any mains wiring is ONLY done by the low-voltage/fire-alarm technician; ESP32s just read volt-free contacts.

9 Commission.

Run every line of `docs/TEST_CHECKLIST.md`: trip each sensor for real, pull a node's power and watch `studio_ready` hold, test SOS from Amma's phone, test a delivery OTP on the door screen. Not “done” until every box ticks.

PART 2 — The two door screens Waveshare 10.1" + Pi Zero 2 W kiosk · ~40 min each

1 Flash the Pi Zero 2 W.

Raspberry Pi Imager → **Raspberry Pi OS (64-bit) Desktop** → gear icon: hostname `door-panel-1` (and `-2`), user + password, Wi-Fi, SSH on.

2 Connect.

Mini-HDMI → the Waveshare's HDMI; the Waveshare's touch USB → the Zero's USB port; power both. The screen works out of the box at 1920×1200.

3 Kiosk mode.

Autostart Chromium full-screen on the panel URL (Claude runs this over SSH):

```
mkdir -p ~/.config/autostart && cat > ~/.config/autostart/panel.desktop <<'EOF'
[Desktop Entry]
Type=Application
```

```
Name=AanganPanel
Exec=chromium-browser --kiosk --noerrdialogs --disable-session-crashed-bubble
https://studio-command.vercel.app/#/display/front-house
EOF
```

Panel 2 uses `/#/display/front-studio`. Disable screen blanking in `raspi-config` → Display. Once the House Pi is live, the same app served on the LAN keeps panels working even when internet is down.

4 Mount.

Front door: the Displays page assigns it “Door sign” — it shows COME ON IN / RECORDING and takes over with the delivery OTP + your standing directions. Studio door: “Studio state” — ON AIR sign + the close-the-door nudge. The old iPad remains the third, in-house panel.

PART 3 — Music Pi built in PARALLEL with the house · separate order & invoice

The complete order-to-first-note build (Raspberry Pi DAC Pro + XLR board + Pianoteq Pro, every connection, every command, 16-point test) is its own document: **PIANO_PI_BUILD_PLAN.html** in the repo's `docs/`. Both projects run together: while robu ships the sensors and boards, the House Pi gets Home Assistant (Part 1) and the owned Pi 5 gets its OS flashed and Pianoteq activated — the DAC Pro just slots on when it arrives. One correction Claude will make at build time: the repo's setup script still writes the old HiFiBerry overlay — it changes to the DAC Pro (`iquaudio-dacplus`) family.

PART 4 — The Idea Menu everything the two Pis (+ Apple TV) can become · tick what you want, Claude builds it into the app

Studio & House

IDEA	WHAT IT DOES	NEEDS	IN THE APP	STATUS
Family SOS	One hold on any phone raises the whole house; wall panels show who needs help	nothing new	<code>/#/sos</code> + Safety page	SHIPPED
Delivery OTP + directions	2-tap hand-off; OTP huge on the door screen with standing directions	door screens	Displays page	SHIPPED
Close-the-door nudge	Door open + room loud → panels and phones ask for the door	door reeds + dB meter	banner everywhere	SHIPPED
Apple TV = HomeKit hub	HA's HomeKit Bridge exposes Aangan to iOS: “Hey Siri, is the studio ready?”, states as scenes in the Home app, family presence automations — the Apple TV is the always-home hub that makes Siri work from anywhere	Apple TV (owned)	works alongside the app	NEXT

Calendar-driven states	Class slots in Google Calendar auto-set the dial to Class and back — no human forgets the sign	nothing	Command page automation	NEXT
Tarang on the studio TV	MIDI-reactive visuals (the existing Tarang engine) on a wall TV during classes and tours; the dial's Class state turns it on	HDMI from a Pi to the TV	a “Visuals” toggle on Command	IDEA
Fleet health card	One card answers “is everything in the school alive?” — Macs, Pis, printers, router (nsm-health feeds it)	nothing	new Home card	IDEA
Voice paging	“Class starting in 5” spoken/shown on the wall panels from the app	panel speakers optional	Displays page button	IDEA
Guest QR at the door	Visitor scans a QR on the door screen → their phone shows live wait status + “come in softly” etiquette	nothing	door panel + tiny page	IDEA
Energy monitor	Per-circuit power on House Pulse — what the studio actually costs per month, AC left-on alerts	CT-clamp energy meter (electrician)	House Pulse	IDEA
Doorbell cam v2	ESP32-CAM upgraded to a proper camera with person detection (Frigate) — snapshot in history + on phones	a better camera later	Safety page	IDEA
Laundry / fridge nudges	Spare SW-420 says “washing machine done”; reed + DHT22 says “fridge door open”	owned spares	history + notifications	IDEA
Rain-on-the-balcony alert	Owned leak sensor outside → “bring the clothes in” on every panel	owned spare	notification	IDEA

Music rig

Honest note: the DAC Pro is output-only (that's why it sounds so good for its job). So music-Pi ideas are MIDI + playback ideas — anything needing audio *inputs* stays on the Mac/console side.

IDEA	WHAT IT DOES	NEEDS	IN THE APP	STATUS
MIDI black-box	The rig silently records EVERY note ever played as MIDI (tiny files, 24/7). An improvisation is never lost — replay any moment through Pianoteq, or export it into a lesson	nothing new	Piano Rig card: “last takes” list + replay cue	NEXT
Practice stats	The black-box becomes numbers: minutes played per day, streaks, most-practised keys/tempo — for Jason AND students	black-box	Piano Rig card graph	IDEA

Preset picker	Full Pianoteq instrument list in the app (not just next/prev) — a NY Steinway for class, a Rhody for the evening, from any phone	nothing	Piano Rig card	NEXT
NSM sample player	sfizz on the rig plays OUR libraries (Salamander + any SFZ we build on the Mac) beside Pianoteq — tanpura drones, pads, harmonium	nothing new	Piano Rig: sound source switch	IDEA
Tanpura + metronome on the rig	Drone in any key + the Groove Metronome on the rig's touch screen — the practice companion lives where the piano is	nothing	rig touch UI	IDEA
Layer/split mode	Piano + pad layered, or bass/piano split — one keyboard, a fuller stage sound	sample player	rig touch UI	IDEA
AirPlay into the console	The rig is an AirPlay/Spotify Connect receiver — any phone plays reference tracks through the XLR outs	nothing	Piano Rig toggle	IDEA
REAPER remote	The rig's touch screen gets a Mac mode: transport, record-arm, marker drop — control the recording Mac from the piano bench	nothing	rig touch UI	IDEA
Nightly REAPER backup	3am rsync of the Mac's REAPER projects to the rig's USB SSD — an off-Mac copy always exists	USB SSD (already owned)	Fleet card shows “last backup”	NEXT
Recording tally	Rec states already cue the rig; the rig screen shows a red ON AIR tally at the piano	nothing	already wired	SHIPPED

How approval works: tell Claude which rows to build (“build the MIDI black-box and the Apple TV HomeKit bridge”) — SHIPPED means live today, NEXT means ready to build immediately, IDEA means a short build after the hardware lands.